

Mission Control: Node/Availability Zone Replacement procedure

Overview

This section describes how to restore consistency after Usage database single node or availability zone (rack) failure or outage lasting longer than hint window size. The procedure involves decommissioning old node(s), creating new node(s) to replace them with empty disks and streaming the data from other replicas. The procedure has different behavior depending on the IP address assignment to new nodes. IP-address assignment is controlled by the underlying platform and is outside the control of Mission Control and the operator; whether the new node reuses the previous IP or gets a new one cannot be chosen by the user.

Node replacement with different IP address

If the IP address is not reused, the new node starts accepting write transactions from coordinator as soon as it joins the cluster. However it does not accept client connections or participate in the read requests. The read consistency is ensured by the other replicas. The new node initiates bootstrap procedure, which is a process of discovering token ranges this node is responsible for and streaming related data from other replicas. Once the streaming operation is complete, the new node has all required data. It then starts accepting client connections and handles all transaction types.

Node replacement with reused IP address

If the IP address is reused the new node is treated as the same node rejoining the cluster. Similarly, it initiates bootstrap procedure, however it does not accept new write transactions. The consistency is meant to be achieved by replaying the hints collected in the coordinators. Once the streaming operation is complete, the node accepts client connection and starts participating in all transaction types. All outstanding hints are replayed to new node.

Initiating node replacement procedure

In order to replace a single node the following `CassandraTask` should be created. In this example we configured the task to replace a single node named `db-east-rack1-sts-0` in `east` datacenter.

Note: `maxConcurrentPods` is only available in Mission Control 1.18 and later. On earlier versions the task replaces one node at a time, regardless of whether the field is set.

```
apiVersion: control.k8ssandra.io/v1alpha1
kind: CassandraTask
metadata:
  name: replace-node
  namespace: ads-db
spec:
  concurrencyPolicy: Forbid
  datacenter:
    name: east
    namespace: ads-db
  jobs:
    - name: replace-node
      command: replacenode
      args:
        pod_name: db-east-rack1-sts-0
  maxConcurrentPods: 1
```

Initiating rack replacement procedure

In order to replace all nodes in the rack the following `CassandraTask` should be created. In this example we configured the task to replace a single node at a time. The testing of the procedure under BAU traffic showed that replacing more than one node in parallel in a 3-node rack greatly increases write latency, which led to write TPS degradation. It is expected that more than one node could be rebuilt at a time in clusters with higher node count; however, the number of concurrently replaced nodes should not be greater than a third of the nodes in a rack.

Note: The Mission Control UI only supports replacing a single node. Whole-rack replacement must be driven via the `CassandraTask` shown below.

```
apiVersion: control.k8ssandra.io/v1alpha1
kind: CassandraTask
metadata:
  name: replace-rack
  namespace: ads-db
spec:
  concurrencyPolicy: Forbid
  datacenter:
    name: east
    namespace: ads-db
  jobs:
    - name: replace-rack
      command: replacenode
      args:
        rack: rack1
  maxConcurrentPods: 1
```

Monitor replacement progress

The progress of node replacement can be monitored by inspecting the relevant CassandraTask. It gives the high level view of completed nodes and the nodes in progress. The output shown below is limited to status for brevity.

```

Status:
Conditions:
  Last Transition Time: 2026-04-29T15:16:41Z
  Message:
  Reason:              Running
  Status:              True
  Type:               Running
Pod Statuses:
  db-dc1-rack1-sts-0:
    Completion Time: 2026-04-28T15:05:56Z
    Start Time:      2026-04-28T05:15:41Z
    Status:          COMPLETED
  db-dc1-rack1-sts-1:
    Completion Time: 2026-04-29T00:31:16Z
    Start Time:      2026-04-28T15:06:01Z
    Status:          COMPLETED
  db-dc1-rack1-sts-2:
    Start Time:      2026-04-29T00:31:21Z
    Status:          RUNNING
Start Time:          2026-04-28T05:15:41Z
Succeeded:           3
Events:              <none>

```

The progress of the rebuild in each node can be checked with the `nodetool netstats` command as shown below. It reports progress for each source endpoint separately giving percentage and volume of already streamed and total data.

```

dse@db-east-rack1-sts-0:~$ nodetool netstats | awk '/\s+\%/([0-9]{1,3}\.){3}/
/10.1.8.127 : 0.0186096%          0.358676/1927.37GB
/10.1.8.164 : 0.00512277%         0.111914/2184.64GB
/10.1.8.173 : 0.01235903%         0.236621/1914.56GB
/10.1.8.191 : 0.0231946%          0.413935/1784.62GB
/10.1.8.204 : 0.0127195%          0.248485/1953.58GB
/10.1.8.233 : 0.0185481%          0.243199/1311.18GB

```